

Week 11 – Interview Preparation Questions – Angular Track

1. What are directives in Angular?

Directives are classes that modify the behavior or appearance of DOM elements. They allow developers to extend HTML with custom functionality.

2. What are the different types of directives?

Three types:

- Component directives (with templates)
 - Structural directives (e.g., `*ngIf`, `*ngFor`)
 - Attribute directives (e.g., `ngClass`, `ngStyle`)
-

3. What is the difference between structural and attribute directives?

Structural directives change the DOM layout (add/remove elements), while attribute directives modify the appearance or behavior of existing elements.

*4. How does `ngIf` differ from `[hidden]`?

`*ngIf` removes/adds elements from the DOM, while `[hidden]` only toggles visibility using CSS.

5. What is a custom directive, and when would you use one?

A custom directive is user-defined to add reusable behavior (e.g., highlighting elements, custom validation).

6. What are pipes in Angular?

Pipes transform data in templates (e.g., date formatting, uppercase conversion).

7. What is the difference between pure and impure pipes?

Pure pipes run only when input changes (better performance), while impure pipes run on every change detection cycle.

8. How do you create a custom pipe?

By using the `@Pipe` decorator and implementing the `transform()` method.

9. What is the async pipe?

It subscribes to Observables or Promises and automatically handles subscription and unsubscription.

10. Why should pipes be preferred over methods in templates?

Pipes are optimized for performance and avoid repeated function execution during change detection.

11. What are the two types of forms in Angular?

Template-driven forms and reactive forms.

12. What is the difference between template-driven and reactive forms?

Template-driven forms rely on HTML templates and directives, while reactive forms are model-driven and defined in TypeScript.

13. What is FormControl in reactive forms?

It represents a single input field and tracks its value and validation status.

14. What is FormGroup?

A collection of FormControls grouped together to manage form state as a whole.

15. What are validators in Angular forms?

Functions that check form control values (e.g., required, minLength, pattern).

16. What is the difference between setValue() and patchValue()?

setValue() updates all fields and requires exact structure; patchValue() updates partial fields.

17. What is form state tracking?

Angular tracks states like touched, dirty, valid, invalid to manage validation and UI feedback.

18. What is two-way data binding in forms?

Using [(ngModel)] to bind input fields with component data, enabling automatic updates in both directions.

19. What is a custom validator?

A user-defined function to implement complex validation logic beyond built-in validators.

20. How do you handle form submission in Angular?

By binding to the (ngSubmit) event and processing form data in the component.

21. What is Dependency Injection (DI) in Angular?

Dependency Injection is a design pattern where a class receives its dependencies from an

external source rather than creating them itself. This improves modularity, testability, and maintainability.

22. What is a service in Angular?

A service is a reusable class that contains business logic, data access, or shared functionality that can be injected into components or other services.

23. How does Angular's DI system work?

Angular uses an injector to create and manage service instances. Dependencies are registered with providers and injected into constructors when needed.

24. What are providers in Angular?

Providers tell Angular how to create or deliver a dependency (e.g., a service instance).

25. What is the difference between providedIn: 'root' and registering providers in a module?

providedIn: 'root' makes the service a singleton across the app, while module-level providers can create scoped instances.

26. What is hierarchical dependency injection?

Angular has a hierarchy of injectors (root, module, component). Services can have different instances depending on where they are provided.

27. What are singleton services, and why are they useful?

Singleton services have only one instance throughout the app, ensuring shared state and efficient resource usage.

28. What is the use of InjectionToken?

It allows injecting non-class dependencies like configuration values or interfaces.

29. How do you test services in Angular?

Using TestBed to configure a testing module and inject services, often with mocks or spies.

30. What is lazy loading in relation to services?

When modules are lazy-loaded, services provided in them may get separate instances, affecting scope and lifecycle.

31. What is routing in Angular?

Routing enables navigation between different views or components in a single-page application without reloading the page.

32. What is a RouterModule?

It is a module that provides routing functionality and allows defining application routes.

33. What is a route configuration?

A mapping between URL paths and components that Angular uses to display views.

34. What is router-outlet?

A directive that acts as a placeholder where routed components are displayed.

35. What is the difference between routerLink and href?

routerLink is Angular's directive for navigation without page reload, while href triggers a full page reload.

36. What is lazy loading in routing?

Loading feature modules only when their routes are accessed, improving initial load time.

37. What are route guards?

They control access to routes (e.g., authentication, authorization) before navigation occurs.

38. What is ActivatedRoute?

A service that provides access to route parameters, query parameters, and route data.

39. What is the difference between query parameters and route parameters?

Route parameters are part of the URL path, while query parameters are optional key-value pairs appended after '?'.

40. How do you handle navigation programmatically?

By using the Router service and calling methods like `navigate()` or `navigateByUrl()`.

41. What are some advanced features of the Angular Router?

Features include lazy loading, route guards, resolvers, child routes, auxiliary routes, and preloading strategies.

42. What is a route resolver?

A resolver pre-fetches data before a route is activated, ensuring the component loads with required data already available.

43. What are child routes?

Child routes allow nesting of routes within a parent route, enabling hierarchical navigation structures.

44. What is a wildcard route?

A route defined with `**` that catches undefined URLs, often used for 404 pages.

45. What is route preloading?

Preloading loads lazy modules in the background after the app loads, improving navigation speed.

46. What is the difference between `navigate()` and `navigateByUrl()`?

`navigate()` uses an array of route segments, while `navigateByUrl()` uses a full URL string.

47. What are auxiliary routes?

They allow multiple routes to be displayed simultaneously in different outlets.

48. What is route reuse strategy?

It controls whether components should be reused or recreated when navigating between routes.

49. What is scroll position restoration in routing?

It restores the scroll position when navigating back or forward between routes.

50. What is the purpose of router events?

Router events allow tracking of navigation lifecycle events like start, end, cancel, or error.

51. What is `HttpClient` in Angular?

`HttpClient` is a service used to make HTTP requests (GET, POST, PUT, DELETE) to interact with APIs.

52. What module is required to use `HttpClient`?

You must import `HttpClientModule` in your application module.

53. What is an Observable in HTTP requests?

HttpClient methods return Observables, allowing asynchronous data handling and stream processing.

54. How do you handle errors in HTTP requests?

Using RxJS operators like `catchError()` or handling errors in the `subscribe()` method.

55. What are HTTP interceptors?

Interceptors allow modifying requests or responses globally (e.g., adding auth tokens, logging).

56. What is the difference between Promise and Observable?

Observables can emit multiple values over time and support operators, while Promises resolve once.

57. How do you send headers in an HTTP request?

By using `HttpHeaders` and passing them in the request options.

58. What is REST API integration in Angular?

It involves consuming backend services using HTTP methods to perform CRUD operations.

59. What is the role of RxJS in HttpClient?

RxJS provides operators for handling asynchronous streams, transforming data, and managing side effects.

60. How do you optimize API calls in Angular applications?

Use caching, debouncing, request cancellation, and efficient state management to reduce unnecessary calls.

61. What is state management, and why is it important in Angular applications?

State management is the practice of managing and synchronizing application data (state) across components. It becomes crucial in large applications to avoid inconsistent data, improve scalability, and simplify debugging.

62. What are different approaches to state management in Angular?

Approaches include local component state, shared services with Observables, and external libraries like NgRx or Akita.

63. What is NgRx, and how does it work?

NgRx is a Redux-inspired state management library using a unidirectional data flow with actions, reducers, selectors, and effects to manage application state predictably.

64. What are actions, reducers, and selectors in NgRx?

- Actions: Describe events or changes
 - Reducers: Define how state changes in response to actions
 - Selectors: Extract specific pieces of state for components
-

65. What are effects in NgRx?

Effects handle side effects like API calls, separating business logic from components and reducers.

66. What is the advantage of using a store over services for state management?

A store provides a centralized, predictable state with better debugging, immutability, and time-travel debugging, unlike services which can become harder to manage at scale.

67. What is immutability in state management?

Immutability means state cannot be modified directly; instead, new copies are created with updated values, ensuring predictable state transitions.

68. What are selectors, and why are they important?

Selectors are optimized functions used to retrieve slices of state efficiently, reducing unnecessary re-renders.

69. What challenges can arise with state management libraries?

Increased complexity, boilerplate code, learning curve, and over-engineering for small applications.

70. When should you avoid using NgRx?

For small or simple applications where state is minimal, using NgRx may add unnecessary complexity.

71. What is RxJS?

RxJS is a library for reactive programming using Observables to handle asynchronous data streams.

72. What is an Observable?

An Observable is a stream of data that can emit multiple values over time, either synchronously or asynchronously.

73. What is the difference between Observable and Subject?

An Observable is unicast by default, while a Subject is multicast and can emit values to multiple subscribers.

74. What are the different types of Subjects?

BehaviorSubject (holds latest value), ReplaySubject (replays previous values), and AsyncSubject (emits last value on completion).

75. What are operators in RxJS?

Operators are functions used to transform, filter, and combine streams (e.g., map, filter, switchMap).

76. What is switchMap, and when is it used?

switchMap cancels previous subscriptions and switches to a new Observable, commonly used in HTTP requests to avoid race conditions.

77. What is the difference between mergeMap, concatMap, and switchMap?

- mergeMap: runs multiple Observables concurrently
 - concatMap: runs sequentially
 - switchMap: cancels previous and switches to latest
-

78. What is error handling in RxJS?

Handled using operators like catchError, retry, and fallback strategies within streams.

79. What is the role of unsubscribe in Angular?

Unsubscribing prevents memory leaks by cleaning up Observables when components are destroyed.

80. How do you manage multiple subscriptions effectively?

Using techniques like takeUntil, async pipe, or subscription management patterns to avoid leaks and improve readability.

81. Why is testing important in Angular applications?

Testing ensures application reliability, catches bugs early, improves code quality, and makes refactoring safer.

82. What are the main types of testing in Angular?

Unit testing, integration testing, and end-to-end (E2E) testing.

83. What tools are commonly used for Angular testing?

Angular uses frameworks like Jasmine for writing tests and Karma for running them.

84. What is TestBed in Angular?

TestBed is a utility that configures and initializes an Angular testing module, simulating the Angular environment for components and services.

85. What is a component fixture?

A fixture is a wrapper around a component instance that provides access to the DOM and component properties for testing.

86. What is the difference between `async()` and `fakeAsync()`?

`async()` handles asynchronous operations automatically, while `fakeAsync()` simulates asynchronous behavior using a synchronous style with `tick()`.

87. What is `tick()` in Angular testing?

`tick()` simulates the passage of time in `fakeAsync` tests, allowing you to resolve asynchronous operations like timers.

88. What are spies in testing?

Spies are used to track function calls and simulate behavior, often used to mock dependencies.

89. How do you test services in Angular?

By injecting them into TestBed and using spies or mocks to test their behavior independently.

90. What is dependency mocking?

Replacing real dependencies with fake or mock versions to isolate the unit under test.

91. How do you test HTTP requests in Angular?

Using `HttpClientTestingModule` and `HttpTestingController` to mock and verify HTTP calls.

92. What is code coverage?

A metric that shows how much of your code is executed during testing, helping identify untested areas.

93. What is end-to-end (E2E) testing?

E2E testing verifies the complete application flow from user interaction to backend integration.

94. What tools are used for E2E testing?

Tools like Protractor (deprecated) and Cypress are commonly used.

95. What is the Arrange-Act-Assert pattern?

A testing structure where you set up conditions (Arrange), execute code (Act), and verify results (Assert).

96. What is shallow testing?

Testing a component without rendering its child components, focusing only on its own logic.

97. What are test doubles?

Mock objects, stubs, or fakes used to simulate real dependencies in tests.

98. How do you test form validation in Angular?

By setting form values programmatically and checking validation states like valid, invalid, touched, and errors.

99. What are best practices for Angular testing?

Write small, isolated tests, mock dependencies, avoid testing implementation details, and ensure maintainability.

100. What is continuous testing in Angular projects?

Running tests automatically in CI/CD pipelines to ensure code quality and prevent regressions.